

Mock Design: WhatsApp / Chat System

Phase 4 · Interview-Ready · Estimated time: 2 hours

1. Concept From First Principles: Applying the Framework

Designing a chat system is one of the most commonly asked HLD problems precisely because it forces you to combine real-time delivery, offline handling, and data modeling for both 1:1 and group conversations — a genuinely broad surface area for a single problem. This is also the first mock design chapter in this Phase to lean heavily on Phase 3 Chapter 9's real-time delivery techniques, so treat it as a direct continuation of that chapter's material, applied under interview conditions.

Step 1 — Clarify requirements

- Functional: 1:1 messaging; group chat (clarify max group size — WhatsApp allows up to a few hundred); message delivery status (sent/delivered/read); online/offline presence.
- Non-functional: messages must not be lost, even if the recipient is offline when sent; low latency for online users; the system should handle a huge number of long-lived persistent connections simultaneously.

Step 2 — Estimate scale

Assume 500 million daily active users, each sending ~40 messages/day, giving ~20 billion messages/day, roughly 230,000 messages/second average and several times that at peak (e.g., New Year's Eve message spikes are a famously real operational event for messaging platforms). Each active user also needs a persistent connection open to receive real-time messages — at 500 million concurrent potential connections, this alone drives major architectural decisions before a single message is even sent.

Step 3 — API / protocol

Sending a message isn't a simple request/response HTTP call — it needs a persistent, bidirectional channel so the server can push new messages to the client without the client polling. Use **WebSockets** (Phase 3, Ch 9) for this: client opens one long-lived WebSocket connection to a chat server on app start, then sends/receives messages over that same connection. Core message shape: {senderId, conversationId, content, clientId, timestamp} — the clientId, generated client-side, is what makes retries safe (Phase 2, Ch 9's idempotency, applied here to prevent duplicate-send on network retry).

Step 4 — High-level design

Client → Load Balancer → **Connection/Gateway Servers**, each holding many open WebSocket connections and maintaining an in-memory map of which users are connected to which specific server instance. When user A sends a message to user B: the message hits A's connection server, which writes it to a durable **Message Store** (a wide-column store like Cassandra, Phase 3 Ch 5, chosen for its write-heavy append-friendly LSM-tree design) and simultaneously looks up which connection server B is attached to (via a shared presence registry, e.g., Redis) and pushes the message there for

immediate delivery if B is online. If B is offline, the message waits durably in the store and is delivered on B's next connect — the store write must happen regardless of B's online status, since durability, not just real-time delivery, is a hard requirement.

Step 5 — Deep dive: delivery guarantees and group chat fan-out

Delivery guarantees: "messages must not be lost" implies at-least-once delivery, which in turn requires the client-generated `clientId` for de-duplication on the receiving end — if a client retries a send after a network blip without confirmation, the server must recognize the duplicate and not deliver it twice. Delivery status (sent/delivered/read) is modeled as separate acknowledgment events flowing back from recipient to sender, each itself needing the same durability treatment, since a lost "delivered" ack shouldn't be treated as a lost message.

Group chat fan-out is the harder half of this deep dive, and directly reuses Phase 4 Chapter 8's fan-out-on-write vs. fan-out-on-read framing before that chapter is even reached — worth naming explicitly if you've studied both. For a group of, say, 200 members, a naive design writes the message once and fans it out to all 200 members' connection servers synchronously on send, which is simple but means one slow/offline member's handling shouldn't block the other 199. The standard answer: write the message once to the durable store, then **fan out asynchronously** via a message queue (Phase 2, Ch 12) to each member's delivery path independently — online members get near-instant push delivery, offline members' copies simply wait in the store for their next connect, and no single member's state affects any other's delivery latency.

Step 6 — Trade-offs and wrap-up

Volunteer: connection servers holding in-memory presence state means a connection server crash drops all its attached users' live connections — clients must detect this and reconnect (typically to a different server via the load balancer), re-registering their presence. Mention end-to-end encryption (as WhatsApp famously implements) as a security consideration worth flagging even if not deep-diving it, tying back to Phase 3 Ch 12's security-at-scale chapter.

2. Real-World Example / Case Study

WhatsApp's own publicly discussed architecture historically used Erlang specifically for its strength in handling millions of concurrent lightweight connections — a direct real-world validation of this chapter's emphasis on connection-handling as a first-class scale concern, distinct from message-processing throughput alone. This exact problem ("design WhatsApp" or "design a chat system") is one of the most frequently repeated HLD prompts across Careem-, Talabat-, and Indian Tier-1 interviews specifically because it tests such a broad slice of real-time systems knowledge in one question.

3. Diagram

```

[User A] <==WebSocket==> [Connection Server 1]
[User B] <==WebSocket==> [Connection Server 2]

A sends message to B:
[Conn Server 1] --write(durable)--> [Message Store (Cassandra)]
                  --lookup B's server--> [Presence Registry (Redis)]
                  --push if online-----> [Conn Server 2] --> [User B]

Group chat (200 members), async fan-out:
[Sender] -> [Message Store: write once]
              |
              [Message Queue]
            /   |   \
    [Member 1] [Member 2] ... [Member 200]
    (each delivered independently; no blocking on others)

Offline member: message waits in store, delivered on next connect.

```

4. Trade-offs Table

Decision	Alternative	Why this choice
WebSockets for client connection	HTTP polling	Persistent bidirectional channel avoids polling overhead/latency at this scale
Cassandra (wide-column) message store	Relational database	Write-heavy, append-friendly LSM-tree design fits chat's access pattern
Async queue-based group fan-out	Synchronous fan-out to all members	One slow/offline member can't block delivery to the rest of the group
Client-generated clientMessageId	Server-generated ID only	Enables de-duplication on client retry, required for at-least-once delivery

5. Common Interview Questions

Q1: "How does a message get delivered if the recipient is offline?"

Model approach: State that the durable store write happens regardless of recipient online status, and delivery to an offline recipient simply happens on their next connect — durability and real-time delivery are two separate concerns, not one.

Q2: "How do you scale group chat to a group with hundreds of members without slowing down delivery?"

Model approach: Explain asynchronous, per-member fan-out via a message queue after a single durable write — explicitly contrast this with synchronous fan-out's blocking risk, and note this previews the fan-out-on-write pattern Chapter 8 covers in more depth for feed systems.

Q3: "How do you prevent a message from being delivered twice if the client retries a failed send?"

Model approach: Name the client-generated clientMessageId directly and explain the server de-duplicates on it — this is Phase 2 Ch 9's idempotency key pattern applied specifically to chat message sends.

6. Practice Exercises

- 1 Run the full six-step framework for "design WhatsApp," out loud, with a timer.
- 2 Design the presence registry: what data does it store per user, and how does it stay correct when a connection server crashes without a clean disconnect?
- 3 Extend the design to support read receipts across a group chat — what changes in the data model and delivery path?

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Designing WhatsApp: Mock System Design Interview"	Full live mock interview walkthrough matching this chapter's structure.
English	"System design mock interview: 'Design WhatsApp or Telegram' (with ex-Google EM)"	Covers delivery guarantees and group fan-out from an engineering-manager perspective.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering real-time chat system design as part of the broader curriculum.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Hindi roadmap covering WebSockets and real-time delivery fundamentals this chapter builds on.

8. Key Takeaways

- Chat systems need a persistent bidirectional channel (WebSockets), not request/response HTTP, for real-time delivery.
- Durability (writing to a message store) and real-time delivery (pushing to an online recipient) are separate concerns handled independently.
- Client-generated message IDs enable de-duplication, required for safe at-least-once delivery under retries.
- Group chat fan-out should be asynchronous per-member after a single durable write, so no one member's state blocks others.
- Connection servers hold in-memory presence state, making connection-server failure and reconnection a real design concern to volunteer.
- This problem's broad surface area (real-time delivery, offline handling, group fan-out, data modeling) is exactly why it's asked so frequently.
- The group fan-out pattern here directly previews Chapter 8's fan-out-on-write vs. fan-out-on-read framing for feed systems.